# Semana 2 24/02

EBNF es un documento que explica la gramática.
**Compilador:** Texto (Lenguaje) → Binario
**Transpilador:** Texto (Lenguaje x) → Texto (Lenguaje y)
**Intérprete:** Texto → Ejecución
**Emulador:** Binario → Binario

Bootstraping: Puentear.

Compilador 1 y varias pasadas:
**El flujo típico es: Scanning→ Parser → AST → Checker (semántica) → IR (lowering) → Optimización → Generación de código**.

- **Primero hace la exploración o scanning**
    - Primero el texto le hace una revisión con el EBNF, que esté bien escrito que tenga sentido en sintaxis.
    - Luego lo divide en componentes léxicos y lo pone en una lista
- Ánálisis (Parser)
    - Agarra esa lista y hace un árbol de sintaxis abstracta, bota cosas que no nos sirve (comas, puntos, parentesis, etc). Como el árbol tiene la estructura, no nos sirven esas cosas.
    - Que la estructura esté bien hecha.
- Decorador (Checker)
    - Una vez tenemos el árbol, el decorador revisa la semántica, revisa tipos, scope, llamadas a funciones, etc. Si no cuadra lanza errores semánticos.
- Optimización:
    - Una fase que mejora el programa para que sea: más rápido, pequeño, y a veces menos consumo de energía o memoria.
    - Por ejemplo hace:
        - Eliminar código muerto.
        - Constant folding: 3*4 → 12
        - Simplificar expresiones: x+0 → x
        - Evitar cálculos repetidos (reutiliza resultados)
        - Mover cosas fuera de loops si no cambian.
- Generación:
    - La fase que convierte el Árbol de Sintaxis Abstracta (AST) o Representación Intermedia (IR) al código de destino. Incluye cosas como
        - Elegir instrucciones reales
        - Decidir registros y manejar spills a memoria
        - Calcular direcciones/stack
        - Emitir el ensamblador final o binario.

**Planteamiento Lenguaje:**
Idea y ejemplos

# Semana 2 26/02

## Gramática libre de contexto:

- Son un montón de reglas de esta forma:
  - [nombre de regla] ::= [por lo que reemplazamos la regla]
  - Solo se reemplaza la regla directamente, por eso se llama libre de contexto
  - Ponemos al menos en el curso, las reglas en mayúscula.
- Por ejemplo:
  - A ::= patito B juliana
  - B ::= corre | baila
- Con esto podemos generar un lenguaje o podemos procesarlo, podemos interpretarlo. Entonces si tengo un texto tipo:
  - 'Patito danza juliana'
  - Podemos aplicarle la gramática para ver si corresponde con la gramática. Como ahí dice danza, entonces ahí ya hay un error, diría que hay un error en el token, componente o lexéma, #2.
  - Se debería poder ver de una forma como de árbol.
- Tenemos otro ejemplo que es para direcciones de personas:
  - Siempre empezamos con una regla principal, que es de donde empezamos a procesar. Vamos a procesar de arriba hacia abajo.
  - Se puede hacer recursividad en la gramática

Direccion ::= Persona ; Casa; Provincia

Persona ::= Nombre Apellido

Nombre ::= Identificador

Apellido ::= Identificador

Casa ::= NuroCasa NombreCalle

NúmeroCasa ::= NumeroEntero

NombreCalle ::= Identificador

Provincia ::= "Cartago" | "Alajuela" | "Heredia" | ...

NumeroEntero ::= ("1" | "2" | "3" | ...) NumeroEntero
| ("1" | "2" | "3" | ...)

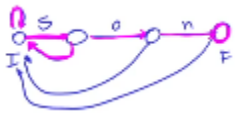
Identificador ::= ("a" | "b" | "c" | "d" | ...) Identificador
| ("a" | "b" | "c" | "d" | ...)

Ocupan una condición de parada (obvio)

Pueden ser recursivas

## Expresiones Regulares

- Es un mecanismo para búsqueda y extracción de texto.
- Es una secuencia de caracteres que representa un patrón
- Ejemplo:
  - Tenemos el texto: “Estas son las mañanitas”
  - El patrón es “Son”
  - En python, la libreria se llama re. Entonces usamos ese patrón en el texto, y devuelve un tipo que puede devolver o None, significando que no encontró o sino:
    - Inicio: Donde inicia la coincidencia
    - Fin: Donde termina la coincidencia
    - Texto: “son”
  - Pregunta: La libreria que hace si encuentra varias coincidencias, el inicia y fin que sería por ejemplo.
- Las expresiones regulares el mecanismo es un “Automata de estados finitos”

- Tenemos un estado De inicio, y vamos viendo que si tenemos esta letra, entonces pasa al siguiente estado. Si ya no lo tiene, vuelve al estado de inicio, y así.
- 


- Comodines:
  - . → cualquier carácter excepto salto de línea, normalmente.
  - ? → el elemento anterior es opcional: aparece 0 o 1 vez.
  - + → el elemento anterior aparece 1 o más veces.
  - * → el elemento anterior aparece 0 o más veces.
  - | → o lógico entre patrones.
  - Ejemplo: gato|perro
  - ^ → inicio de línea o inicio del texto.
  - $ → fin de línea o fin del texto.
  - [] → clase de caracteres; acepta uno de los que están dentro. O también para intervales
  - Ejemplo: [abc] = a o b o c
  - Ejemplo: [0-9], [A-Z], [a-z]
  - () → agrupa partes del patrón y también puede capturar coincidencias.
  - Ejemplo: (ab)+
- Ejemplos Comodines:
  - Si quiero buscar todas las palabras relacionada con mesa digamos:
    - ‘mes.*’
    - Mes, y cualquier carácter excepto salto de línea luego aparece 0 o más veces.
  - Para hola con varias a:
    - ‘Hola+’
    - Acepta Hola con una o varias A

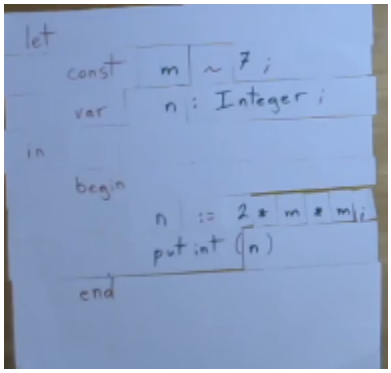
## Grámaticas EBNF:

- Podemos incluir en las grámaticas el Regex.
  - `Entero ::= (-|\+)?[0-9]+`
  - Ahí es que puede hacer 0 o 1 vez - o +. Luego 0-9 parece 1 o más veces.

    - Cuando queremos referirnos a digamos uno de los carácteres especiales de forma literal, tenemos que usar un backslash antes.
    - `Flotante ::= (-|\+)?([0-9]+)?\.[0-9]+`
-

## Introducción al explorador

Un explorador detecta componentes léxicos.

- Recibe el archivo, primero usa la gramática, no usa toda, sino los símbolos terminales. Agarra primero digamos los comentarios, entonces lo quita. Luego separa por lexemas o tokens.
- 

- Primero separa el let y le da atributos, como la línea y columna en donde estaba. Porque a partir de esos atributos podemos retornar errores después.
    - Digamos agarra el let, y le asigna que es una palabra clave
    - Luego lo mismo con el const
    - Agarra la m, que aún no sabe que es una variable, entonces por ahora es solo un identificador, y revisa si es válido.
    - El ~ es asignación, es una palabra clave.
    - El 7 le asigna como entero
    - El ; le asigna palabra clave
    - Var → palabra clave
- Dentro del componente léxico se divide por
    - Lexema: la palabra que estaba dentro
    - Un tipo
    - Y una serie de atributos, por ejemplo lo de la línea.

Luego el siguiente paso es subir toda la lista que estamos generando y la va comparando contra la gramática, y además subiendo a un árbol.

## Gramáticas LL

Las gramáticas que nosotros podemos procesar. Para nosotros poder escribir un compilador, la gramática no puede tener ambigüedad (al menos la nuestra). Una vez construimos una gramática, vamos a una parte llamada análisis sintáctico, que es la que verifica los tokens, la gramática, etc.

- Hay algoritmos que pueden verificar cualquier gramática pero son muy complicados, entonces al menos en nuestro caso no lo vamos a usar (Cocke-Younger-Kasami ó Early).
- Lo que usamos son analizadores sintacticos o compiladores que usen algoritmos descendentes o ascendentes. Nos vamos a concentrar en un algoritmo descendente, que es el algoritmo por descenso recursivo.
- Implementan una gramática LL o LR. Nos concentramos en LL, las LR son más complicadas de implementar, a mano incluso demasiado complicado las LR, necesitaríamos herramientas automatizadas, como flex o bison.
- Toman una lista de tokens y analiza cada token de izquierda a derecha uno por uno. Con base a eso construye el Árbol de Sintaxis Abstracta.

Recordemos de las gramáticas (no solo LL y LR). Bajo la sintaxis EBNF, tenemos:
Regla ::= contenido
La raíz del árbol es siempre esa regla principal o regla programa.
Reglas de la gramática:

- Una gramática no puede ser ambigüa, si yo llego a un determinado lugar, yo tengo que saber cual es la regla que está ahí.
- Para el algoritmo por descenso recursivo, no pueden tener recursividad por la izquierda, porque causa que se encicle. Por ejemplo:
    - A ::= Aa | b
    - Convertimos reglas a métodos:
    - ```
      A() {
        A()
        verifique(a)
        ó
        verifique(b)
      }
      ```

Vamos a convertir la grámatica en una gramática LL(1):

- L: left
- L: derivación por la izquierda
- (1): cantidad de tokens que revisamos para tomar decisiones (ver cual es la siguiente regla o lo siguiente que voy a procesar).
- Cumple 2 reglas particulares:
    - Si la gramática contiene una regla con la forma X|Y, X y Y deben iniciar con tokens distintos (no ambigüedad).
    - Si la gramática contiene una expresión de tipo X*, X debe iniciar con un Conjunto de tokens que pueda seguir a X* en este contexto particular.

Ejemplos:

- Factorización Izquierda
    - Regla ::= XY | XZ
    - Pasa a
    - Regla ::= X(Y|Z)
      ```

- Recursión Izquierda:
    - Regla ::= X | Regla Z
    - Pasa a
    - Regla ::= X(Z)*
    - 

Cumplen con dos reglas particulares:

1. Si la gramática contiene una regla con la forma X | Y, X y Y deben iniciar con tokens distintos.

2. Si la gramática contiene una expresión de tipo X*, X debe iniciar con un conjunto de tokens que pueda seguir a X* en este contexto particular.

- 

# Semana 3 03/03

Trabajos del proyecto

# Semana 4 10/03

Ssh -J [serveo.net](serveo.net) usuario@profe_aurelio
Rsync para subir cosas.

# Semana 4 12/03

## Analizador

Tenemos la siguiente gramática:

Función ::= (Comentario)? mae Identificador (Parámetros)
{Instruccion+}

Parámetros ::= Valor (/ Valor)+

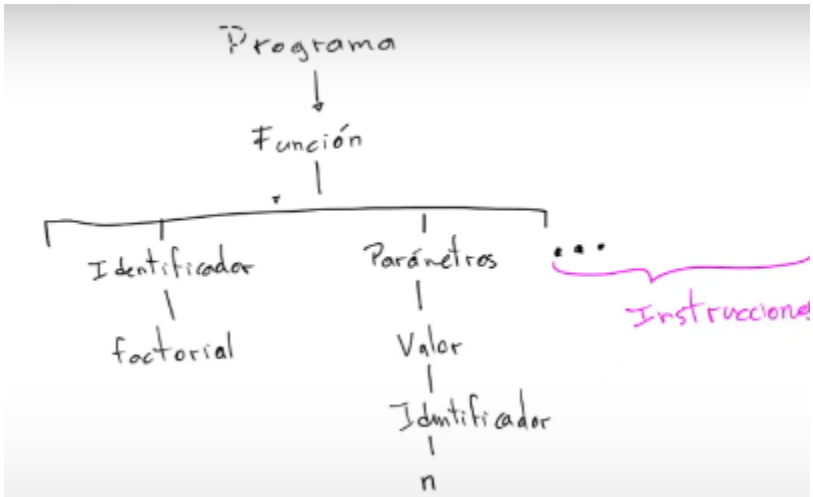
Y tenemos un código de factorial.

El explorador primero separa en lexemas el código, digamos

| | |
|---|---|
| mae | Palabra clave |
| factorial | Identificador |
| ( | Puntuación |
| n | Identificador |
| ) | Puntuación |
| { | Puntuación |
| … | … |
| } | Puntuación |

Bajando vamos a ir creando la estructura del árbol para poder procesarlo y hacer estructuras más avanzadas.

La cosa es que ya sabiendo que eso es una función que se llama factorial, con parámetro n y con x instrucciones, ya no necesitamos la palabra mae, los paréntesis, ni las llaves. Entonces si al árbol le quitamos eso que no necesitamos, se vuelve en un **árbol de análisis abstracto**.




Como voy a lograr esto, yo voy a tener un objeto **Analizador**

```
Analizador (listaComponentes)
      analizar() : ASA
      // Analizar va a llamar al primer nivel de la gramática que es
analizar programa.


analizarPrograma()
      …
      analizarFuncion()
```

```
analizarFuncion()
//Verifica los tokens en orden que aparece
     verificar('mae')
//compara el lexema del componente léxico actual, con el parámetro
     analizar-identificador()
//Revisa si sigue un identificador
     verificar('(')
     analizar-parámetros()
     verificar(')')
     verificar('{')
     verificarInstrucción()
     Mientras componenteActual sea el inicio de una Instrucción
          verificarInstrucción()
```

La función verificar compara el lexema del componente léxico actual, con el parámetro, si es el mismo, entonces todo está bien y pasa. Si es distinto da un error en la estructura.
Luego

La forma más facil de hacer el árbol es que sea:

- Nodo
    - Tipo → enum
    - Contenido → String
    - Hijos → Lista

## Manejo de errores

Tipos de errores:

- **Léxicos:** Cuando escribimos mal un identificador o una palabra clave dentro del texto.
- **Sintácticos:** Los errores que detecta el analizador, por ejemplo una expresión matemática con paréntesis mal equilibrados (le falta o sobra un parentesis de un lado).
- **Semánticos:** Cuando aplicamos un operador de números a string por ejemplo. O cuando ponemos un + entre un número y un string
- **Lógicos:** Por ejemplo recursividad infinita, ya error dentro del código.

**Gramáticas LL y LR:**
Tienen una propiedad que se llama de **prefijo viable**, o sea son capaces de encontrar un error de forma inmediata solo fijándose en el siguiente componente léxico. Al menos para errores léxicos y sintácticos.

**Reporte de errores:**
Tiene que tener ciertas características vitales:

- Claro y conciso.

- Informar el número de línea y columna.
- De ser posible debería sugerir como solucionarlo

Cuando nos encontramos un error, en el resto del código pueden haber:
- Errores encadenados producto de ese error.
- Otros errores hechos por el programador.

Idealmente deberíamos ser capaces de encontrar el error principal y tratar de no reportar todos los errores secundarios que pasaron por el principal.

**Estrategias para manejo de errores:**
- **Modo pánico:** Me topo un error, me brinco todo hasta encontrar un componente léxico de sincronización.
    - Un componente de sincronización es un componente léxico en donde sabemos más o menos que va a andar bien y así.
        - Por ejemplo un fin de línea, me brinco todo hasta el fin de línea porque seguramente las cosas de esa línea pueden hacer que salten errores secundarios.
        - Palabras claves
        - Puntos y coma
- **Nivel de frase:** El compilador reporta el error y repara el error. No puede reparar todos los errores, pero los que podamos.
    - Por ejemplo, encontró una coma donde hay un punto y coma. Entonces reporta eso, pero lo repara y lo tira con punto y coma
- **Producciones de error / Reglas EBNF de error:** Hay errores que son tan comunes en el código, que podríamos agregar una regla con la estructura de esos errores comunes. Entonces eso lo “repara”, y debería también sugerir la estructura correcta.
    - Es bueno pero requiere algo de experiencia del lenguaje para poder saber esos errores comunes. Además podría hacer una gramática gigante, y por ende el analizador y explorador también.
- **Corrección global:** Serie de algoritmos que puede hacer esas producciones de error en caliente. Pero es muy difícil de implementar. Encuentra un error, y calcula en caliente el mínimo número de cambios para poder corregirla. Además podría hacer el compilador lento.

## Análisis por descenso recursivo

Revisa la gramática aprovechandose de la recursividad.

```python
def analizar(self):
    """
    Método principal que inicia el análisis siguiendo el esquema de
    análisis por descenso recursivo
    """
    self.__analizar_programa()
```

```python
def __analizar_programa(self):
    """
    Programa ::= (Comentario | Asignación | Función)* Principal
    """

    # Los comentarios me los paso por el quinto forro del pantalón (y
    # de todos modos el Explorador ni siquiera los guarda como
    # componentes léxicos)

    # pueden venir múltiples asignaciones o funciones
    while (True):

        # Si es asignación
        if self.componente_actual.tipo == TipoComponente.IDENTIFICADOR:
            self.__analizar_asignación()

        # Si es función
        elif (self.componente_actual.texto == 'mae'):
            self.__analizar_función()
```

Si lo que encuentra es un identificador, entonces significa que está haciendo una asignación.
Si por otro lado encuentra mae, enconces es porque hay una función.

```python
        else:
            break

    # De fijo al final una función principal
    if (self.componente_actual.texto in ['jefe', 'jefa']):
        self.__analizar_principal()
    else:
        raise SyntaxError(str(self.componente_actual))
```

Si encuentra algo más, entonces break
Y al final el main, que se representa por jefe o jefa. Y ahí no hay manejo de errores, ahi solo tira una excepción y ya.

```python
def __analizar_parámetros(self):
    """
    Parametros ::= Valor (/ Valor)+
    """
    # Fijo un valor tiene que haber
    self.__analizar_valor()

    while( self.componente_actual.texto == '/'):
        self.__verificar('/')
        self.__analizar_valor()

    # Esto funciona con lógica al verrís... Si no revienta con error
    # asumimos que todo bien y seguimos.
```

Un valor si o si tiene que haber, entonces primero analiza eso.
Luego revisa si hay '/' que son como las comas en este lenguaje. Si lo hay entonces lo verifica y analiza el siguiente valor, y eso está dentro de un while.
El verificar lo que hace es consumir el token, y compararlo con lo que tiene de parámetro verificar.

```python
def __analizar_instrucción(self):
    """
    Instrucción ::= (Repetición | Bifurcación | Asignación | Invocación | Retorno | Error | Comentario )

    Acá hay un error en la gramática por que no reconoce las
    Invocaciones por la falta de corregir un error en la gramática LL

    Invocación y Asignación ... ambas dos inician con un Identificador
    y acá no se sabe por cuál empezar.
    ...
    La solución en código que yo presentó acá esta sería como algo así

    Instrucción ::= (Repetición | Bifurcación | (Asignación | Invocación) | Retorno | Error | Comentario )

                                                ^                     ^

    Ojo los paréntesis extra                    |                     |
    """
```

Identificador
↓
ambigüo

```python
# Acá todo con if por que son opcionales
if self.componente_actual.texto == 'upee':
    self.__analizar_repetición()

elif self.componente_actual.texto == 'diay siii':
    self.__analizar_bifurcación()

elif self.componente_actual.tipo == TipoComponente.IDENTIFICADOR:

    if self.__componente_venidero().texto == 'metale':
        self.__analizar_asignación()
    else:
        self.__analizar_invocación()

elif self.componente_actual.texto == 'sarpe':
    self.__analizar_retorno()

else: # Muy apropiado el chiste de ir a revisar si tiene error al último.
    self.__analizar_error()

# Ignorado el comentario
```

Upee es repeticion
Diay siii es bifurcación
Luego revisa si es un identificador, y dentro de este revisa que más tiene, pero no debería ser así. Tendriamos que arreglar la gramatica para que eso no pase.
Sarpe es retorno
Y error en este caso es como los throws y eso en este lenguaje
Lo que falta en realidad es eso, de manejo de errores verdadero.

```python
def __analizar_bloque_instrucciones(self):
    """
    Este es nuevo y me lo inventé para simplicicar un poco el código...
    correspondería actualizar la gramática si lo consideran necesario.

    BloqueInstrucciones ::= { Instrucción+ }
    """
```

```
# Obligatorio
self.__verificar('{')

# mínimo una
self.__analizar_instrucción()

# Acá todo puede venir uno o más
while self.componente_actual.texto in ['upee', 'diay siii', 'sarpe'] \
        or self.componente_actual.tipo == TipoComponente.IDENTIFICADOR:

    self.__analizar_instrucción()

# Obligatorio
self.__verificar('}')
```

Basado en todo lo anterior se podría asumir más o menos como sirve esto.

### Construcción de un Árbol de Sintaxis Abstracta (ASA)

Que video de mierda, no se ver ese video de aurelio

## Semana 5 24/03

**Progra, entregar jueves después de semana santa:** Entrega del explorador. Hace un split del texto y crea una lista de tokens/lexemas, a esos les agrega información adicional para crear la lista (tipo, lexema, atributos).

- ¿Qué errores debería encontrar el explorador?
    - Caracteres desconocidos.
- Archivo de sintaxis de colores para puntos extra